Individual Report

Hotel Management System (HMS)

Module: UFCF8S-30-2 Advanced Software Development

Student Name	Ahmed Shaain Firaq
Student ID	S2401269
Role	Person 1 — Backend Developer + UML Lead

1. Introduction

This report reflects on my individual contribution to the Hotel Management System (HMS) project, developed as part of the Advanced Software Development module at Vila College. The system is a full-stack web application built using ASP.NET Core (C#) for the backend, Angular for the frontend, and PostgreSQL as the database. As Person 1, I was responsible for the complete backend architecture, database design, security implementation, UML diagrams and backend system documentation.

2. Individual Contribution

My primary responsibility was designing and building the complete backend system from scratch. The work began with setting up the GitHub repository, defining the branching strategy and scaffolding the ASP.NET Core Web API project with all necessary packages including Entity Framework Core, Npgsql, JWT Bearer authentication and BCrypt.

I designed and implemented all eight C# entity models — User, Hotel, Room, Booking, AncillaryService, BookingService, Payment and AuditLog — and configured the AppDbContext with all relationships, foreign key constraints and decimal precision settings using EF Core Fluent API. I then generated and applied the initial EF Core migration which automatically created all eight database tables in PostgreSQL.

On the security side, I implemented JWT token generation and validation, BCrypt password hashing, Role-Based Access Control middleware enforcing least privilege across four roles (Guest, Staff, Manager, Admin), session timeout middleware and an account lockout system that locks accounts after five consecutive failed login attempts for 15 minutes. I also built a comprehensive audit logging service that records all key system actions including logins, bookings, check-ins, check-outs, payments and cancellations.

The business logic I implemented included a cancellation fee engine applying the 14-day, 3-day and 72-hour cancellation rules specified in the requirements, peak and off-peak room pricing logic based on check-in month, room availability checking across overlapping booking dates to

prevent double bookings and a payment service calculating final bills including ancillary service totals.

I wrote all five API controllers — AuthController, RoomController, BookingController, ReportController and AuditController — exposing REST endpoints consumed by the Angular frontend. I also created the database seeder populating two hotels, twenty rooms, four ancillary services and five test user accounts across all roles, and configured CORS to allow Angular requests from port 4200.

For the UML documentation required in Part I, I produced five diagrams — a use case diagram covering all four actors and their interactions, a class diagram reflecting all eight entities with attributes, methods and visibility, and three sequence diagrams covering the booking flow, check-in flow and cancellation flow. Additionally, an ER diagram was produced showing all database tables with primary and foreign keys.

3. Reflection on Learning

This project significantly deepened my understanding of building production-oriented backend systems. Working with ASP.NET Core 9 and Entity Framework Core taught me the value of clean layered architecture — separating models, services and controllers kept the codebase maintainable as complexity grew. Implementing JWT authentication from scratch provided a deeper understanding than working with pre-configured authentication libraries alone, giving me full insight into token claims, bearer schemes and expiry management.

The database work with PostgreSQL and Npgsql reinforced the value of a code-first migration approach — being able to define the schema in C# and have EF Core generate and apply it automatically saved significant time and reduced the risk of schema inconsistencies. I also learned the importance of seeding realistic test data early, as it allowed all endpoints to be tested meaningfully via Swagger throughout development rather than waiting until the frontend was ready.

Working as part of a two-person team taught me the importance of clear role separation and communication. Defining responsibilities at the outset meant both team members could work in parallel without blocking each other. Git with a feature branch workflow proved essential for maintaining a clean and always-functional main branch throughout the project.

4. Challenges Faced

PostgreSQL Configuration

One of the more complex technical challenges encountered was configuring the PostgreSQL database connection during initial setup. The development environment had two versions of PostgreSQL installed (17 and 18), and identifying which version was actively listening on port 5432 required investigation using system tools. Once the correct instance was identified, the connection string and authentication settings were configured accordingly. This experience provided practical exposure to database server administration and environment configuration —

skills that are highly relevant in professional development environments where multiple database versions may coexist.

Package Version Management

Managing NuGet package compatibility presented a significant challenge. The project targeted .NET 9 however the NuGet package manager defaulted to resolving the latest versions which were only compatible with .NET 10. This required a systematic approach to dependency management — explicitly specifying compatible version numbers for each package and understanding the dependency chains between EF Core, Npgsql and the JWT Bearer package. This deepened my understanding of how package ecosystems work and the importance of version pinning in professional projects.

Scope Management Under Time Constraints

Delivering a complete, working system within a two-person team required disciplined scope management. Rather than attempting to implement every feature at full complexity, deliberate decisions were made to prioritise the core functionality and security requirements while simplifying lower-priority features such as analytics visualisations. This reflects the kind of pragmatic decision-making required in real Agile development environments where delivery within constraints is essential.

5. Emerging Technologies

This project provided exposure to several emerging development practices and technologies. The rise of AI-assisted development tools such as GitHub Copilot represents a significant shift in how software is developed professionally. Evaluating and understanding these tools forms an important part of modern software engineering practice, and their growing adoption across the industry makes familiarity with them increasingly valuable.

Entity Framework Core's code-first migration approach represents a modern shift away from manual SQL schema management, enabling database schema to be version-controlled alongside application code. This approach is now widely adopted in enterprise .NET development and reflects current industry best practices.

JWT-based stateless authentication is now the industry standard for securing REST APIs in distributed systems, replacing traditional session-based approaches. Its use in this project reflects real-world API security patterns used across modern web applications. BCrypt for password hashing with an adaptive cost factor directly aligns with OWASP password security guidelines and provides protection against brute-force attacks even as hardware capabilities improve.

Finally, the implementation of Role-Based Access Control enforcing least privilege principles directly addresses the secure development requirements outlined in the module specification and reflects security-first design practices increasingly mandated in professional software development environments.